

Domande sicurezza 2020-2021

by zeta.exe, alesios99 & pulapemasa

Intro

Come valuto la giusta difesa?

La giusta difesa è valutata proporzionando: **Integrità, confidenzialità e disponibilità** con particolare riguardo verso il controllo degli accessi e l'autenticazione/non ripudio.

Per valutare ciò si fa un calcolo del tipo: minacce * valore * vulnerabilità (???)

Confidenzialità: fa riferimento al fatto che una risorsa (o un servizio) deve essere accessibile solo a chi è autorizzato ad usufruirne. Può essere descritto anche come **capacità di controllare/assicurare la privacy/riservatezza delle informazioni** rispetto a soggetti non autorizzati. Questo concetto non si applica solo, per esempio, al contenuto di un file, ma anche alla conoscenza dell'esistenza di tale file o meno o anche all'utilizzo di un certo mezzo (ad esempio la rete).

Integrità: fa riferimento al concetto di assicurazione che un'informazione non sia stata modificata e che quindi la risorsa sia fruibile nel modo esatto nel quale è stata resa disponibile. Può essere descritto anche come **capacità di controllare/assicurare la non modificabilità/attendibilità** da parte di soggetti non autorizzati delle informazioni.

Disponibilità (Availability): fa riferimento al fatto che se una risorsa deve essere accessibile a degli utenti, un attacco non deve inibire questo accesso (esempio un attacco DDoS).

Può essere descritto anche come capacità di **controllare/assicurare la possibilità di usare risorse e servizi** da parte degli utenti autorizzati.

Alcuni esempi di attacchi alla disponibilità sono il crash di un sistema, la manomissione di algoritmi di scheduling per non far mai eseguire un certo processo o anche la distruzione fisica di un computer

Descrivere gli obiettivi dell'informatica e fare un esempio di loro violazione

(Credo si riferisca ai criteri CIA, anche perché obiettivi dell'informatica è un po' vago)

Gli obiettivi di sicurezza dell'informatica da tenere a mente nello sviluppo di un applicativo sono i cosiddetti criteri CIA: **Confidentiality, Integrity e Availability**.

Con il concetto di **confidenzialità** si intende tutto ciò che concerne la **visibilità delle informazioni**.

Questo comprende sia il contenuto delle informazioni sensibili, ma anche la sola conoscenza di esse: un malintenzionato può trarre vantaggio dal sistema attaccato anche solo essendo al corrente dell'esistenza di certi documenti sensibili. Un esempio di violazione di confidenzialità (disclosure) è l'invio delle password in chiaro a un database. Un attaccante può appropriarsene sia accedendo al database, che sfruttando attacchi man-in-the-middle.

Con **integrità** si intende la **consistenza di un intero applicativo**: un attaccante può modificare i file all'interno di un sistema o il sistema stesso (ad esempio per inibire qualche controllo di input).

Con availability si definisce la **disponibilità di un servizio quando un utente lo richiede**. L'availability può essere violata con un attacco di tipo DOS (Denial Of Service).

Crittografia

Considerando l'applicazione della crittografia ASIMMETRICA in combinazione con MESSAGE DIGEST per la realizzazione di firme digitali efficienti, e considerando l'utente A che possiede la coppia di chiavi A_priv e A_pub, si descrivano:

- gli step che A deve seguire per inviare un file F firmato all'utente B
 - gli step che l'utente B deve seguire per convalidare la veridicità della firma quando riceve il file F
- 1) Il **mittente crea**, dato il messaggio che intende inviare, **il digest** con un algoritmo qualsiasi (tipo MD5), in seguito cripta tale digest con la sua chiave privata (tutti possono leggere decriptando con la sua chiave pubblica, ma nessuno può modificare e ri-crittare il digest) ottenendo così la firma digitale.
 - 2) **Invia** il messaggio e il digest criptato.
 - 3) Il **destinatario**, una volta ricevuto il messaggio, **autentica il digest** (decifra il message digest mediante la chiave pubblica del mittente) ottenendo così il digest originale.
 - 4) Dato il messaggio in chiaro, **applica la funzione digest e ottiene un message digest** che andrà a confrontare con quello inviato dal mittente insieme al messaggio:
 - Se **combaciano** il **messaggio non ha subito alterazioni**
 - Se **non combaciano** qualcuno ha effettuato **modifiche non autorizzate** al messaggio trasmesso

Si spieghi il funzionamento della crittografia a chiave pubblica, indicando in particolare di come si può usarla per implementare firme digitali (in realtà basta anche la risposta sopra)

Nei sistemi asimmetrici a chiave pubblica ho **due chiavi**, una pubblica (disponibile a coloro a cui si fornisce) e una privata (di mio possesso); **i messaggi criptati con una delle due chiavi possono essere decriptati con l'altra**. Per implementare le firme digitali, posso **criptare il messaggio con la mia chiave privata** così che **il mittente usando la mia chiave pubblica possa confermare la provenienza del messaggio e possa essere sicuro che sono io il mittente e che nessuno ha modificato il messaggio** (non importa se qualcuno l'abbia visto o meno).

Solitamente però gli algoritmi a chiave pubblica sono poco efficienti in termini computazionali e quindi, per realizzare firme digitali, si preferisce creare prima un digest del messaggio tramite un algoritmo di hashing come ad esempio MD5 (operazione molto rapida) e successivamente crittare il digest con la chiave privata del mittente, che risulta molto più veloce rispetto a crittare il messaggio originale.

Vengono poi inviati al destinatario sia il messaggio che il digest cifrato; il destinatario può confermare che sono io il mittente decriptando il digest con la mia chiave pubblica e può controllare che il messaggio sia integro calcolando a sua volta il digest del messaggio e confrontandolo con quello decriptato.

Challenge/response con chiave asimmetrica

Il meccanismo challenge/response sfrutta una **funzione f**, conosciuta dall'utente, per verificarne l'identità. Il sistema fornisce un **valore x** all'utente che, per essere autenticato, dovrà **calcolare f(x)** e **inviare il risultato al sistema**.

Questa funzione f può essere una qualsiasi funzione matematica, tuttavia spesso si tratta di una **funzione crittografica a chiave asimmetrica**; in questo caso il sistema fornisce il valore all'utente che, per dimostrare di conoscere la funzione, cripterà il valore con la chiave asimmetrica (che potrebbe essere ad esempio la sua password di autenticazione) e invierà il risultato al sistema, che applicando **la medesima funzione crittografica otterrà x se l'utente ha risposto correttamente**.

Per evitare che un attaccante possa ascoltare la conversazione e ottenere f(x) si utilizzano i **nonce**; **il valore x generato dal sistema cambia ad ogni richiesta**, in modo che anche f(x) sia diversa.

Si descriva come si possono combinare crittografia a chiave asimmetrica e algoritmi message digest per ottenere meccanismi efficienti di firma digitale

La crittografia a chiave pubblica soddisfa i **requisiti necessari per la firma digitale** (integrità e non ripudio) in maniera **indipendente dal requisito della confidenzialità**. La firma digitale quindi utilizza un meccanismo a chiave pubblica, utilizzare solo RSA per garantire integrità e non ripudio e confidenzialità del messaggio risulta essere costoso, quindi si sacrifica la confidenzialità (se possibile) e si firma il message digest del messaggio, sostanzialmente si applica RSA al digest. Firmare l'intero messaggio è molto costoso, quindi **si firma il digest**.

Quindi, definendo i punti:

- 1) Il **mittente crea**, dato il messaggio che intende inviare, **il digest** con un algoritmo qualsiasi (tipo MD5), in seguito cripta tale digest mediante la sua chiave privata ottenendo così la firma digitale.
- 2) **Invia** il messaggio.
- 3) Il **destinatario**, una volta ricevuto il messaggio, **autentica il digest** (decifra il message digest mediante la chiave pubblica del mittente) ottenendo così il digest originale.
- 4) Dato il messaggio in chiaro, **applica la funzione digest e ottiene un message digest** che andrà a confrontare con quello inviato dal mittente insieme al messaggio:
 - Se **combaciano il messaggio non ha subito alterazioni**
 - Se **non combaciano** qualcuno ha effettuato **modifiche non autorizzate** al messaggio trasmesso

Risposta breve: Il destinatario ha la certezza di aver ricevuto il messaggio corretto in quanto facendo il digest ha la certezza che non sia stato modificato, seppur cifrato.

Si descriva in breve come si può aumentare l'efficienza dei meccanismi di firma digitale sfruttando la combinazione di RSA e MD5

Essendo RSA molto costoso posso **firmare solo il digest** creato con MD5, così avrò l'**autenticità** del messaggio grazie a RSA e l'**integrità** grazie al digest MD5.

N.B.: A differenza della combinazione RSA + DES, che ottiene il maggior grado di confidenzialità, la combinazione RSA + MD5 garantisce la migliore integrità e autenticità.

È più difficile il brute forcing su password con 3 caratteri alfabetici o con 4 cifre?

Nel primo caso abbiamo $(2 \times 26)^3$ possibilità (2 perché sono maiuscole e minuscole dal momento che non è specificato). Otteniamo circa 140680 possibilità.

Nel secondo caso abbiamo 10^4 possibilità (10000). È quindi meno difficile il secondo caso.

Si spieghino le differenze tra crittografia simmetrica e asimmetrica, e le implicazioni sulla relazione fra numero di chiavi e utenti per ognuno dei due casi

Crittografia simmetrica: Una **sola chiave** per collegamento, quindi sia mittente che destinatario usano la stessa chiave scambiata in precedenza

Crittografia asimmetrica: Sia mittente che destinatario hanno ciascuno una **coppia di chiavi**, una pubblica ed accessibile a chiunque e una privata che ha solo il soggetto in questione, se cripto con la pubblica solo con la privata posso decriptare e viceversa se cripto con quella privata solo la pubblica potrà decriptare il messaggio.

Nella crittografia simmetrica ogni "collegamento" necessita di una chiave segreta, quindi all'aumentare degli utenti da collegare il numero di chiavi da generare cresce esponenzialmente

($N^2 = N(N-1)/2$ chiavi); nella crittografia asimmetrica, invece, è sufficiente generare **una coppia di chiavi per ogni nodo/utente**, e diffondere le chiavi pubbliche. Questo sistema ha una crescita lineare di chiavi generate ($2*N$)

[Risposta lunga]

Gli algoritmi simmetrici a chiave segreta, sono tutti quegli algoritmi i quali fanno uso di **una chiave segreta condivisa tra mittente e destinatario**, lo scambio e quindi la condivisione di tale chiave deve essere effettuata in modo sicuro, altrimenti non risulta possibile garantire l'integrità e la confidenzialità del messaggio trasmesso. La formula che rappresenta tale meccanismo è la seguente: $\text{decrypt}(k, \text{encrypt}(k, p)) = p$

Dove con k intendiamo la chiave, con p il messaggio in chiaro. Uno **svantaggio** dei sistemi con chiave simmetrica è costituito dal fatto che **i tre requisiti** (Confidenzialità, Integrità, Ripudio) devono essere **implementati tutti insieme**, non è possibile implementarli separatamente. Inoltre un altro svantaggio riguarda il numero di chiavi richieste in una rete per permettere l'uso della crittografia da parte di tutti i componenti della stessa, risulta essere necessario avere una chiave per ogni coppia di componenti che intendono comunicare tra di loro. $\text{Numero_k} = n(n - 1)/2$.

Nei sistemi asimmetrici a chiave pubblica ho due chiavi, una pubblica (conosciuta da chi l'ha ricevuta dall'utente owner) e una privata (conosciuta dal suo owner). Posso criptare il messaggio con la mia chiave privata, in questo modo il ricevente avrà la sicurezza che il mittente sono proprio io, inoltre posso criptare il messaggio con la chiave pubblica del destinatario così da assicurare la confidenzialità. Quindi posso fornire confidenzialità e integrità, con conseguenziale autenticazione e non ripudio, in maniera indipendente l'uno dall'altra. Il numero di chiavi private è pari al numero di individui che intendono parlare all'interno della rete.

Cosa fa una funzione di hash? Che caratteristiche deve avere?

La funzione di hash **riassume un testo in pochi byte**. La caratteristica che deve avere è quella di **avere meno collisioni possibili**.

Una funzione di hash è una particolare funzione che caratterizza un messaggio, ottenendo una sorta di checksum crittografico dell'ordine delle centinaia di bit. Questo permette di avere **codici riassuntivi** (e quindi meno pesanti) **dei messaggi ma soprattutto non invertibili**, quantomeno in tempi umani.

La caratteristica ideale per una funzione hash è quella di avere un codice univoco per ogni stringa, ma questo è matematicamente impossibile, per cui si punta ad avere **funzioni con meno collisioni (codici non univoci) possibile**.

Perché la crittografia DES non è più considerata sicura? Quali modi di suo utilizzo potrebbero migliorare la sicurezza?

La crittografia DES non ha un **algoritmo matematico** che ne dimostra l'affidabilità. Inoltre, nel 1977 (e quindi con macchine di quell'epoca) si è riusciti a fare bruteforcing ad un messaggio in 4 mesi, mediante **l'uso in parallelo di circa 3500 computer**.

Un modo per migliorare la sicurezza del DES sarebbe di **crittare ripetutamente il messaggio con chiavi diverse** (es. triple des), oppure usare il nuovo standard AES (**Advanced Encryption System**), basato sul principio di "confusione e diffusione"

Descrivere crittografia end to end e crittografia di collegamento.

La crittografia **end-to-end** implementa una codifica del messaggio da applicazione ad applicazione; il messaggio **non viene decriptato prima di essere effettivamente consegnato**.

Dunque il messaggio è effettivamente confidenziale, tuttavia questo tipo di crittografia non protegge del tutto la confidenzialità; un attaccante può trarre vantaggio anche solo dalla conoscenza di certe comunicazioni: seppur il contenuto dei messaggi sia criptato, **un attaccante può comunque vedere con chi sto parlando** e quando.

Questo tipo di codifica viene applicata dal protocollo SSL/TLS e per poter funzionare il software deve supportare tale implementazione, in particolare devono supportarla i browser, i server web ed eventuali proxy server.

La crittografia di collegamento, anche chiamata **host-to-host**, implementa una codifica del messaggio solo da macchina a macchina; il messaggio viene **decriptato una volta che arriva alla macchina di destinazione**, anche se non è ancora stato consegnato all'applicativo che l'ha richiesto, passando di fatto in chiaro all'interno della macchina. Questo tipo di crittografia, eseguita direttamente dal software di rete, è trasparente alle applicazioni che si comportano come se non ci fosse. Il **protocollo** che implementa questo tipo di crittografia è **IPSec** che funziona in due modalità:

- **Trasporto: cripta tutto a partire dal layer di trasporto** e offre una protezione simile ad SSL ma è appunto host-to-host piuttosto che end-to-end (svantaggio) ma è **completamente trasparente alle applicazioni** (vantaggio)
- **Tunneling** (tipico funzionamento VPN): **cripta anche il layer di rete**, quindi anche gli indirizzi IP originali, sostituendoli con degli **IP dei tunnel endpoints**; in questo modo se un attaccante osserva il traffico, non solo **non può leggere il contenuto** del messaggio poiché criptato, ma **non può nemmeno sapere chi comunica con chi** (solitamente i tunnel endpoints sono i router di confine di due sottoreti o il router di un utente e il router di una sottorete di destinazione, dunque tutto quello che vede l'attaccante è del traffico da un router all'altro, senza poter però sapere quali applicazioni stanno effettivamente comunicando, dunque questa crittografia protegge dall'analisi del traffico). In questa modalità c'è dunque una "perdita" di sicurezza all'interno della sottorete (i pacchetti viaggiano in chiaro fino al tunnel endpoint di partenza), ma una grande sicurezza tra i due endpoint, attraverso la rete non sicura. Questa soluzione viene utilizzata ad esempio per lo smartworking o per chi deve connettersi ad una sottorete privata in sicurezza attraverso internet.

Sicurezza applicazioni

Si descriva il funzionamento di un attacco SQL injection

L' SQL Injection consiste nell'**inserimento di una query SQL** attraverso l'input utente al fine di modificare l'esecuzione di comandi SQL parametrici. Un attacco eseguito con successo può:

- leggere/modificare **dati sensibili**
- eseguire operazioni di **amministrazione del database**
- recuperare il contenuto di un file presente nel **filesystem** del dbms

I problemi di SQL Injection si possono manifestare in tutti i programmi che fanno uso di un DBMS, storicamente sfruttati nel caso di web applications scritte con PHP, Perl...

Questo genere di attacchi sfrutta alcune vulnerabilità di un sistema, come **evitare di fare encoding o convalidazione sull'input dell'utente** che, sfruttando il fatto che i sistemi SQL utilizzano certi caratteri terminatori (come '), permette ad un attaccante di terminare la parte di input del form e

aggiungere dei comandi malevoli.

Fare encoding sull'input (quindi trasformare i caratteri potenzialmente terminatori in una certa codifica, ad esempio trasformando < in <) o fare una validation su di esso (accettarlo o rifiutarlo in base ad una whitelist o ad una blacklist che individuino rispettivamente input corretti ed input malevoli) permette di evitare che l'attaccante possa eseguire altre operazioni oltre al fornire l'input effettivamente richiesto.

Si descriva il funzionamento di un attacco di tipo buffer overflow

Consiste nello scrivere in un buffer **uscendo intenzionalmente dai limiti dello stesso**, andando a scrivere in aree di memoria non assegnategli adiacenti. Questo exploit, dovuto alla mancanza di controlli **OutOfBound** (assente ad esempio in C), permette di modificare, per esempio, i valori di variabili locali, ma anche tutto ciò che c'è nel **activation record** della funzione vulnerabile, tra cui anche il **return address** della funzione, questo perché, mentre lo stack "cresce" verso i valori bassi degli indirizzi di memoria, i buffer crescono verso i valori alti, dunque sopra il buffer nello stack ci sono proprio i registri di configurazione dell'activation record.

Generalmente dunque questo tipo di attacco cerca di **sovrascrivere il return address** delle funzioni in esecuzione, così che punti a una funzione malevola precedentemente inserita dall'attaccante nel buffer stesso.

Il codice malevolo solitamente è uno shellcode, ovvero un programma che, tramite una chiamata di sistema, permette di **sostituire il processo corrente con uno nuovo**, in questo caso con una shell, ed effettuare una syscall per ottenere i privilegi kernel.

Ci sono due difficoltà principali all'uso di questo exploit:

- E' necessario **modificare anche il Base Pointer**, un puntatore che indica la base dello stack e, nel caso di funzioni chiamate, un riferimento al base pointer del chiamante. Questo puntatore viene modificato incondizionatamente durante un attacco di tipo buffer overflow, in quanto l'attaccante scrive in un'area di memoria contigua, non può, ad esempio, modificare solo il return address e lasciare tutto il resto invariato (se potesse farlo non avrebbe più bisogno di fare questo attacco in quanto potrebbe saltare direttamente al codice malevolo, senza cambiare il return address della funzione).
Se il Base Pointer presenta un valore inconsistente, il programma termina. Per individuare il Base Pointer l'attaccante potrebbe estrapolarlo eseguendo in un ambiente di debug il programma e ottenere il base pointer del chiamante, oppure può provare diversi valori finché l'attacco non va a buon fine.
- L'attaccante deve **"indovinare" la posizione esatta del return address**, specie quando si ha a che fare con stack che crescono in maniera non deterministica. L'attaccante può aiutarsi inserendo all'inizio dello shellcode una serie di NOP, ovvero di istruzioni che "non fanno nulla", in modo da poter effettuare un salto impreciso al suo codice malevolo.

Le principali contromisure per questo tipo di attacchi sono la **modifica degli indirizzi di memoria virtuale da un'esecuzione all'altra** o l'introduzione di un **canary value che cambia ad ogni esecuzione** e che viene inserito dopo il Base Pointer; ogni volta che il programma invoca un metodo, dovrà controllare che il canary value non sia cambiato, altrimenti terminerà con un'eccezione. Controllare ogni singolo metodo però può essere molto costoso in termini computazionali, per questo il compilatore gcc offre diversi tipi di protezione di tipo canary value da applicare solo a quelle funzioni che più probabilmente possono soffrire di questa vulnerabilità (ad esempio funzioni con buffer molto grandi o che utilizzano funzioni di allocazione della memoria)

[Risposta alternativa]

Il termine buffer overflow indica una situazione in cui una **scrittura su un buffer esce dai limiti del buffer stesso** "invadendo" dunque un'area di memoria non assegnata a quel buffer ma ad altre entità, in questi casi i valori delle **aree di memoria adiacenti vengono sovrascritti** (ex return address).

Esistono diversi tipi di buffer overflow:

- Memoria virtuale di un processo
- Stack buffer overflow
- Heap overflow

Se opero con codice su rom + heap è possibile avere un buffer overflow?

Nel Buffer Overflow sovrascrivo un pezzo di memoria e a seconda della zona che vado a sovrascrivere posso avere un attacco. Potrei avere nell'heap dati che altero con il Buffer Overflow ma **non posso alterare gli indirizzi di ritorno perché non risiedono nell'heap ma nello stack** che nel caso in esame non è presente.

Quindi, nel caso in esame ho tre fattori da considerare:

- Le istruzioni che non possono essere modificate
- I dati che sono modificabili perché sono sull'heap
- Gli indirizzi di ritorno che non sono modificabili perché sono sullo stack che qui non è presente.

La risposta è quindi: **sì**, posso avere buffer overflow **ma senza modifica degli indirizzi di ritorno**.

AGGIUNTA: se il linguaggio utilizzato è un linguaggio che utilizza puntatori (come ad esempio C), posso utilizzare un puntatore a funzione che risiedendo sull'heap dirotti l'esecuzione su codice creato dall'attaccante.

Se in una funzione scritta in linguaggio C viene fatto overflow su stack con un'istruzione "strcpy" riscrivendo l'indirizzo di ritorno al suo chiamante, ma l'istruzione successiva alla strcpy è scritta in modo da interrompere il programma (ad esempio con "exit()"), cosa accade? Si può avere un completo att buffer overflow?

Strcpy in C è una **funzione** e, come tale, **ha un valore di ritorno** (e quindi anche un indirizzo di ritorno), dunque se l'overflow viene eseguito tramite la strcpy, **il return address modificato è proprio quello della strcpy stessa**. Di conseguenza, una volta terminata la strcpy, il programma salterà al codice malevolo, evitando dunque di eseguire la exit().

Quindi sì, si può avere un completo attacco di tipo buffer overflow.

Come è strutturata la stringa per portare allo stack overflow

In un attacco di tipo stack overflow generalmente l'**attaccante scrive nel buffer** (quindi nelle aree "valide") il suo shellcode malevolo, contornato da NOP (istruzioni che non fanno nulla) in modo da poter fare un salto impreciso. Successivamente andrà a scrivere fuori dal buffer il return address che non è altro che un indirizzo ad una cella del buffer stesso e un base pointer valido, in modo da evitare segmentation faults.

Sicurezza sistemi operativi

Si descriva la differenza tra DAC e MAC facendo un esempio di una politica di controllo degli accessi per ognuno dei due tipi

Nel **Discretionary Access Control** (DAC), il proprietario di una risorsa concede l'accesso ad altri a sua discrezione (es: google docs).

Un altro modello è il **Mandatory Access Control** (MAC), dove il sistema impone un modello che limita e controlla la discrezionalità degli utenti nell'assegnare i diritti di accesso alle risorse (Ex Bell-LaPadula, BIBA → no R/W up, no W/R down).

I sistemi MAC si suddividono in

- **Modelli multi-livello:** ad ogni risorsa e ad ogni utente viene **assegnato un livello** e un utente può accedere ad una risorsa solo se **livello utente $\geq$ livello risorsa**.
Due esempi sono il modello Bell-LaPadula, orientato alla confidenzialità (non permette ad un utente di livello alto di riscrivere informazioni di livello alto ai livelli più bassi) ed il modello Biba, orientato all'integrità (non permette ad un utente di livello alto di riscrivere informazioni di livello più basso al suo livello, non può aumentare l'importanza di un contratto).
- **Modelli Multi-Laterali:** i dati sono controllati in base a **raggruppamenti semantici dei dati**. Un primo esempio è il modello Bell-LaPadula ampliato con compartimenti, dove un utente può accedere ad una risorsa solo se **livello utente $\geq$ livello risorsa && compartimenti utente (= compartimenti risorsa (= contenuto))**.
Un altro modello è la Muraglia Cinese, utile per gestire i conflitti di interesse; in questo modello un utente può accedere ad una risorsa solo se questa fa parte di qualche compartimento al quale l'utente ha già avuto accesso in precedenza o se NON fa parte di un compartimento in conflitto di interesse con i compartimenti ai quali l'utente ha già avuto accesso.

DAC: Il proprietario della risorsa concede l'accesso, per esempio Google Documenti

MAC: Il sistema impone un modello che controlla l'accesso, ad esempio ur mom

Differenza tra mandatory access control e discretionary access control con esempio per ognuno
guarda domanda sopra

Si descriva il funzionamento di una Access Control List e si spieghi il concetto di "mediazione completa" nel contesto dei sistemi operativi sicuri

Mediazione completa significa **controllare ogni accesso**, tramite una access control list, dove, data ogni risorsa, possiamo **inserire chi ha diritto e che diritti ha**, quindi lettura, scrittura o esecuzione.

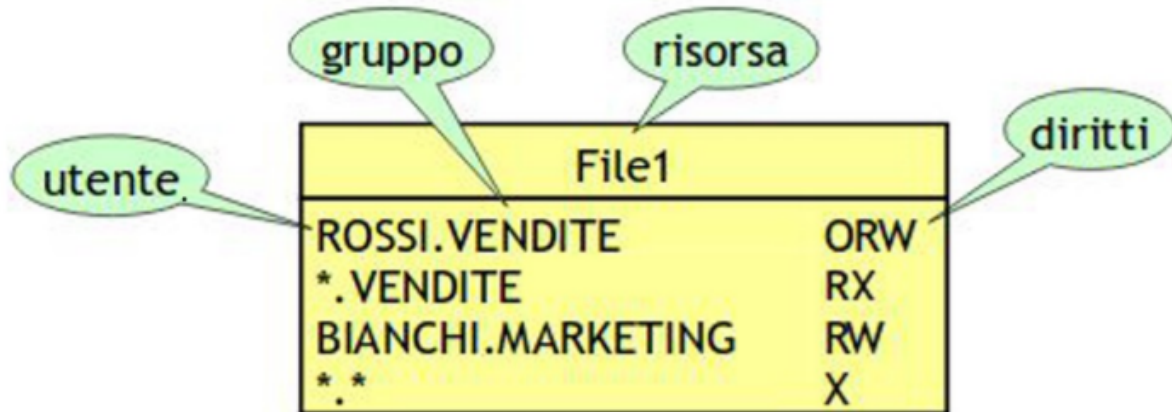
Nei moderni sistemi operativi generalmente questa operazione non viene implementata completamente poiché richiedere l'autenticazione ad ogni operazione ridurrebbe di molto l'usabilità del sistema stesso.

Tramite l'access control list andiamo a **risparmiare in termini di spazio rispetto alla tabella completa**, è facile determinare la lista degli utenti che hanno un diritto specifico su una risorsa condivisa, ma soprattutto è **facile revocare/modificare l'accesso di un utente a una risorsa**.

Unix utilizza un modello ancora più semplificato della ACL, i bit di protezione che richiedono solo 9 bit per file per stabilire i diritti, tuttavia questa soluzione ha un livello di granularità inferiore ad una ACL, poiché possiamo definire permessi solo per l'owner, per un certo gruppo di utenti e per il resto del mondo; non possiamo quindi definire permessi per due gruppi distinti di utenti.

[Risposta alternativa]

L'access control list è una lista dove, data ogni risorsa, possiamo inserire chi ha diritto e che diritti ha, quindi lettura, scrittura o esecuzione. Tramite l'access control list andiamo a risparmiare in termini di spazio rispetto alla tabella completa, è facile determinare la lista degli utenti che hanno un diritto specifico su una risorsa condivisa, ma soprattutto è facile revocare/modificare l'accesso di un utente a una risorsa



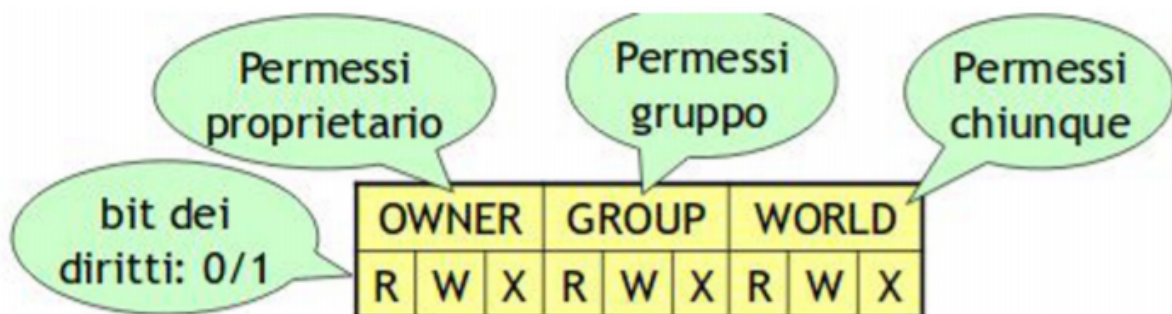
La ACL tuttavia ha un livello di granularità inferiore di una lista completa poiché possiamo definire permessi solo per l'owner, per un certo gruppo di utenti e per il resto del mondo; non possiamo quindi definire permessi per due gruppi distinti di utenti.

Mediazione completa: ogni tentativo di accesso deve essere controllato; in teoria la struttura di un sistema operativo deve controllare la disponibilità dell'autorizzazione ad ogni accesso richiesto, ma la maggior parte dei sistemi esegue il controllo una sola volta;

Si descriva il funzionamento di una access control list basata su 9 bit di protezione come quella usata nei sistemi operativi di tipo Unix/Linux

Unix utilizza un modello ancora più semplificato della ACL, i bit di protezione che richiedono solo 9 bit per file per stabilire i diritti, tuttavia questa soluzione ha un livello di granularità inferiore ad una ACL, poiché possiamo definire permessi solo per l'owner, per un certo gruppo di utenti e per il resto del mondo; non possiamo quindi definire permessi per due gruppi distinti di utenti.

Ci sono dei vantaggi in termini di occupazione di memoria ed inoltre è molto semplice gestire i permessi, il proprietario della risorsa assegna/modifica i diritti di ogni risorsa



R: read, W: write, X: execute

Descrivere il modello Bell-LaPadula completo

Il modello Bell-LaPadula è un **modello MAC di sicurezza multi livello** che nasce in ambito militare per garantire la confidenzialità dei dati. Nei modelli multilivello come questo, sia i soggetti (utenti) che gli oggetti (dati) sono **classificati secondo dei livelli di sicurezza** in modo che un utente possa accedere ad un dato **solo se livello_utente ≥ livello_dato**.

Il modello Bell-LaPadula si occupa di implementare una contromisura contro i covert channel in ambito di confidenzialità implementando due proprietà:

- **Simple Security Property** (no read up): un soggetto non può leggere oggetti di livello più alto.
- **Confinement Property** (no write down): un soggetto non può scrivere oggetti di livello più basso; in questo modo un utente (potenzialmente malevolo) non può trasferire documenti del suo livello a livelli più bassi, eliminando così un possibile covert channel.

Si descriva come funziona una politica di sicurezza secondo il modello di Bell La Padula esteso con compartimenti.

Si dica inoltre se tale modello definisce una politica di sicurezza di tipo MAC o DAC

Il modello di Bell-LaPadula usa uno schema di controllo accessi di tipo MAC che garantisce due proprietà:

- **Simple security property** (no-read-up)
- **Confinement property** (no-write-down)

Grazie alla prima proprietà viene impedito di leggere una risorsa dotata di un livello di sicurezza maggiore rispetto a quello detenuto dall'utente, la seconda dice che un soggetto non può scrivere oggetti di classificazione più bassa (questo per evitare condivisioni non permesse di dati sensibili tra un livello e un'altro).

Il modello di Bell-LaPadula **esteso con compartimenti**, sostanzialmente classifica i livelli di sicurezza per soggetti e oggetti associando le informazioni e i soggetti a compartimenti, i compartimenti possono essere intesi come progetti/missioni/argomenti. L'accesso a una certa risorsa è consentito solo se **il livello del soggetto è maggiore uguale del livello dell'oggetto e i compartimenti del soggetto sono maggiori uguali ai compartimenti dell'oggetto**.

[Risposta alternativa]

Un primo modello multi-lateral è il Bell-LaPadula esteso con compartimenti, sostanzialmente classifica i livelli di sicurezza per soggetti e oggetti associando le informazioni e i soggetti a compartimenti, i compartimenti possono essere intesi come progetti/missioni/argomenti. L'accesso a una certa risorsa è consentito solo se il livello del soggetto è maggiore uguale del livello dell'oggetto e i compartimenti del soggetto sono maggiori uguali ai compartimenti dell'oggetto. Il modello Bell-LaPadula definisce una politica di sicurezza di tipo MAC.

Esempio:

Supponiamo di avere due reparti: IT e contabilità.

Gli utenti assegnati a IT hanno livello di autorizzazione 2, mentre gli utenti di contabilità hanno livello 1. Se il file "fattura.txt" richiede il livello di autorizzazione 1 o superiore, con il Bell-LaPadula classico sia gli utenti IT che contabilità possono accedervi.

Se invece il file definisce che questo sia accessibile solo a chi ha autorizzazione ≥ 1 AND sia assegnato al reparto contabilità; a quel punto gli utenti IT non possono più accedervi a prescindere dal loro livello di autorizzazione.

Si spieghi come la crittografia simmetrica possa essere usata per implementare un algoritmo di autenticazione challenge-response

Negli algoritmi challenge/response la password è in realtà una **funzione matematica**, la crittografia simmetrica può essere usata per implementarlo così:

- il sistema passa all'utente un messaggio in chiaro
- l'utente codifica il messaggio con un algoritmo crittografico e la sua chiave segreta
 - la response sarà positiva solo nel caso in cui il messaggio sarà decodificato con successo

Si descriva la struttura di un pacchetto IP trasmesso usando il protocollo IPsec in modalità tunneling, spiegando il tipo di protezione che si ottiene

Il pacchetto sarà composto da:

- Pacchetto applicativo
- Header TCP
- Header IP originale
- Informazioni protette
- IPsec
- Nuovo header IP (in chiaro)

Grazie alla **modalità tunnelling** il livello al quale opera IPsec si **abbassa ulteriormente** rispetto al modello trasporto, andando al di sotto del livello IP, **cifrando quindi gli indirizzi IP di mittente e destinatario invece che solo i metadati da TCP in sù.**

Chi intercetta il pacchetto potrà quindi solo sapere da quale rete parte il messaggio, ma non dal mittente vero e proprio.

Con IPsec tunnelling abbiamo inoltre una **maggiore resistenza a sniffing e modifica dei messaggi**, traffic analysis e IP spoofing.

[Risposta alternativa]

L'idea alla base di IPsec è quella di **offrire un livello di sicurezza aggiuntivo** lavorando sopra il livello IP, sostanzialmente criptando anche i metadati del livello TCP, questo permette di nascondere il tipo di comunicazione che sta avvenendo a un potenziale attaccante, il quale saprà solo che la macchina A sta parlando con la macchina B.

Sono presenti due modalità per questo protocollo:

- 1) **Trasporto:** vengono criptati tutti i metadati **da TCP compreso in sù.**
- 2) **Tunneling:** il livello a cui opera IPsec si abbassa ulteriormente, posizionandosi al di sotto del livello IP, quindi **si va a cifrare pure gli indirizzi IP del mittente e del destinatario**, inserendo poi un nuovo header IP contenente IP sorgente e destinazione dei router delle reti del mittente e del destinatario.

Per esempio: se ho intenzione di inviare un messaggio a una certa macchina in una certa rete, utilizzando IPsec tunneling mode, cripto il mio indirizzo IP e quello della macchina a cui voglio inviare, al loro posto sarà visibile solo l'indirizzo IP pubblico del mio router e l'indirizzo IP pubblico del router del destinatario, il quale avrà il compito di smistare correttamente il pacchetto ricevuto. Chi intercetta esternamente può sapere solo da quale rete parte il messaggio e a quale rete arriva ma non precisamente a chi. Si va dunque a complicare ulteriormente il meccanismo, rendendolo dunque più difficile da impostare. Viene a crearsi il concetto di VPN, utili per realtà di lavoro remoto o dipartimenti remoti connessi attraverso reti pubbliche. In generale con IPsec abbiamo una maggiore resistenza a sniffing e modifica dei messaggi creando più livelli di crittografia, rendendo più ardua l'operazione di decriptazione. Ho una maggiore protezione contro traffic analysis e una maggiore protezione contro IP spoofing in quanto anche l'indirizzo IP di origine può essere autenticato.

Si descriva la struttura di un pacchetto IP trasmesso usando il protocollo IPsec in modalità trasporto, spiegando che tipo di protezione si ottiene

Il pacchetto sarà composto da:

- Pacchetto applicativo
- Header TCP
- Informazioni protette

- IPsec
- Header IP (in chiaro)

Grazie alla **modalità trasporto** del protocollo IPsec si vuole offrire un livello di sicurezza aggiuntivo lavorando sopra il livello IP andando a **criptare i metadati del livello TCP**.

Chi intercetta il pacchetto non potrà quindi sapere il contenuto del pacchetto, non saprà nemmeno chi sono i veri attori che comunicano (mi pesano le palle a riscrivere, l'ho spiegato in end-to-end prima)

Si descriva la struttura di un pacchetto IP trasmesso usando il protocollo SSL, spiegando che tipo di protezione si ottiene.

SSL (**Secure Socket Layer**), è un protocollo che opera **sopra il livello TCP** (livello di trasporto).

- **Hello Phase:** inizialmente abbiamo una comunicazione durante la quale vengono inizialmente **scambiate varie informazioni**, come la versione del protocollo, un session ID, Cypher suite (setting degli algoritmi crittografici) e parametri di inizializzazione TCP. Il server risponderà positivamente se la connessione risultasse possibile. Questa fase è chiamata Hello Phase.
- **Server Authentication:** il server invia **certificato**, la sua **chiave pubblica**, ed una **richiesta di certificato**. Viene inviata la chiave pubblica perché essa sarà utilizzata per criptare la chiave di crittografia che verrà utilizzata per tutta la comunicazione, il certificato serve per far sì che il client sia sicuro dell'identità del server. Chiave pubblica e certificato non sono criptati (sono di dominio pubblico).
- **Client Authentication:** vengono **inviati dal client il certificato, la session key e la certificate verification**. Il certificato serve al server per essere sicuro dell'identità del client, la session key è la chiave (valore generato in maniera randomica) che verrà utilizzata per la comunicazione che si sta instaurando, e il certificate verification rappresenta la conferma, da parte del client, che il certificato precedentemente inviato è stato verificato. La **session key è criptata con la chiave pubblica del server**.
- **Fine dell'Handshaking:** La quarta e ultima fase, prevede lo **scambio di conferme per quanto riguarda il setting delle impostazioni di sicurezza** definite durante la negoziazione effettuata e un messaggio di finished che indica la fine dell'operazione di handshake e permette la trasmissione dei pacchetti veri e propri data la comunicazione.

Il pacchetto dunque conterrà le sue informazioni originali per quanto riguarda il livello di rete (quindi indirizzi e porte), conterrà poi un header SSL con le informazioni del protocollo e poi il contenuto applicativo vero e proprio viene criptato con la chiave di sessione generata in fase di Handshake. Questo offre una protezione dei dati end-to-end, ovvero da applicazione ad applicazione ma non protegge da attacchi di analisi del traffico: un attaccante può comunque vedere se e quando un soggetto sta comunicando e con chi.

Si spieghi come funziona un firewall "packet filter", e come può essere usato per proteggere una sottorete da attacchi IP-spoofing provenienti dall'esterno della sottorete stessa

Packet filter: il suo livello di azione è il protocollo di rete quindi il livello IP. Può prendere decisioni in base all'indirizzo IP sorgente e/o destinazione ma anche in base alla porta utilizzata per la comunicazione. Può fare **deduzioni riguardanti l'appartenenza di un certo indirizzo IP** (rete interna o esterna), inoltre può prevenire attacchi Dos e IP spoofing. Tutte le decisioni prese da questo tipo

di firewall si basano su una certa tabella, chiamata tabella di filtraggio, la quale, utilizzando per esempio regex, identifica tutti quei indirizzi IP le cui comunicazioni e pacchetti devono essere bloccate. Sinteticamente è un filtro che controlla il traffico dati (= pacchetti IP) da/verso la rete. Per proteggere da IPSpoofing potrebbe per esempio bloccare qualsiasi connessione proveniente dall'esterno con un indirizzo IP interno.

[Risposta alternativa]

Un firewall packet filter **opera a livello di rete**, controllando gli header IP e le porte di comunicazione del traffico passante tra l'esterno e la rete interna, bloccando o consentendo il traffico in base alla configurazione effettuata nella tabella di filtraggio del firewall. Essa può operare con meccanismi di blacklisting (più semplice) o whitelisting (più sicuro). Un firewall di tipo packet-filtering può proteggere una sottorete da attacchi IP-Spoofing controllando se un pacchetto proveniente dall'esterno dichiara un indirizzo IP interno come mittente..

Si descrivano le differenze tra firewall di tipo packet filter e firewall di tipo application gateway

Packet filter: il suo livello di azione è il protocollo di rete quindi il livello IP. Può prendere decisioni in base all'indirizzo IP sorgente e/o destinazione ma anche in base alla porta utilizzata per la comunicazione. Può fare **deduzioni riguardanti l'appartenenza di un certo indirizzo IP** (rete interna o esterna), inoltre può prevenire attacchi Dos e IP spoofing. Tutte le decisioni prese da questo tipo di firewall si basano su una certa tabella, chiamata tabella di filtraggio, la quale, utilizzando per esempio regex, identifica tutti quei indirizzi IP le cui comunicazioni e pacchetti devono essere bloccate. Sinteticamente è un filtro che controlla il traffico dati (= pacchetti IP) da/verso la rete.

Il firewall application gateway, chiamato anche proxy, agisce invece a livello applicativo in simil modo, ovvero, **monitora il traffico tra applicazioni facendo in modo di mettersi in mezzo e ridistribuendo i pacchetti tra le due applicazioni**. Questo tipo di firewall quindi agisce non su pacchetti singoli ma su flussi di pacchetti e permette di controllare i dati applicativi in dettaglio. Si può ad esempio implementare un filtro anti spam con questo tipo di firewall.

[Risposta alternativa]

Il **firewall application gateway**, detto anche Proxy, agisce a livello applicativo per **monitorare il traffico tra applicazioni**. Quindi per comunicare con un'altra applicazione interna alla rete protetta bisogna connettersi al firewall il quale farà da proxy per l'applicazione. A sua volta il firewall si connette all'applicazione interna e replica i pacchetti fra le due connessioni.

Un esempio di firewall di questo tipo può essere un proxy FTP per bloccare i trasferimenti di file in uscita ma non quelli in entrata. A differenza delle tecniche di packet filtering, che consentono di selezionare il traffico in base alle informazioni dei protocolli di trasporto e di rete, i proxy server sono normalmente in grado di prendere decisioni sulle richieste da inoltrare anche in base alle caratteristiche del protocollo applicativo che trattano. Ad esempio, mentre uno screening router può esclusivamente accettare o proibire il traffico FTP, un proxy FTP può anche selezionare il traffico in base ai comandi specifici del protocollo FTP (potrebbe ad esempio essere configurato in maniera tale da consentire richieste di importazione ma non di esportazione di file).

"MAC" e "DAC" possono convivere? Perché?

Sì, si possono applicare ad una risorsa sia MAC che DAC, dando però la precedenza al MAC in quanto **non deve essere possibile ad un utente di modificare delle regole di accesso** mandatorie proprie del sistema. Se da un lato il DAC (Discretionary Access Control) funziona basandosi sul concedere l'accesso ad una risorsa in base a dei gruppi (tipo google docs), dall'altro lato il MAC

(Mandatory Access Control) funziona assegnando dei livelli alle risorse ed agli utenti per potervi accedere. Nulla però impedisce che si possano applicare entrambi ad una risorsa.

Descrivere le due modalità di protezione nei sistemi operativi: ACL e "capabilities". Unix quali usa? e con quali limiti?

non abbiamo fatto capabilities (<https://it.wikipedia.org/wiki/Capability>)

Si descriva il funzionamento di un attacco XSS

XSS è un attacco di tipo code injection, si verifica quando un attaccante riesce a eseguire del codice malevolo all'interno del browser di un utente sfruttando una vulnerabilità di un sito web ignaro dell'attacco, il codice malevolo è implementato in un linguaggio di scripting, ad esempio JavaScript. Uno script è eseguito in un ambiente protetto (browser) che non permette la modifica del sistema ospitante, tuttavia può avere accesso a informazioni sensibili (cookies), può inviare richiesta HTTP contenenti un qualsiasi dato sensibile verso una qualsiasi destinazione. Le conseguenze principali di un attacco XSS possono essere: Furto di cookie, Keylogging, Phishing. XSS è divisibile in tre differenti tipologie:

- **Persistent XSS:** il codice malevolo risiede nel DB del sito web. Lo schema di attacco del Persistent XSS è divisibile in diversi punti:
 - L'attaccante utilizza una form del sito web per inserire del codice malevolo in una pagina web
 - La vittima richiede una pagina dal sito web
 - Il sito web (ignaro) include nella pagina web la stringa inserita dall'attaccante ed invia la risposta alla vittima
 - Il browser della vittima esegue lo script malevolo contenuto nella pagina web
- **Reflected XSS:** il codice malevolo risiede nella richiesta HTTP della vittima. Lo schema di attacco del Reflected XSS è il seguente:
 - L'attaccante crea un URL contenente una stringa malevola e lo invia alla vittima per esempio tramite mail
 - La vittima clicca sull'URL
 - Il sito web invia la risposta ed include la stringa malevola nella risposta (Questo dipende dall'implementazione del sito web, ad esempio un motore di ricerca che mostra la keyword cercata)
 - Il browser della vittima esegue il codice malevolo
- **DOM-based XSS:** la vulnerabilità risiede nel codice eseguito nel client, non nel server. Il DOM-based XSS sfrutta lo scripting dinamico: la capacità di una pagina web di creare il contenuto runtime, anche questa metodologia può essere divisa in più punti:
 - L'attaccante crea un URL contenente una stringa malevola e lo invia alla vittima
 - La vittima clicca sull'URL
 - Il sito web riceve la richiesta, ed invia la risposta senza includere il codice malevolo
 - La pagina web creata dal sito è dinamica, e il codice JS incluso nella pagina
 - Estrae il codice malevolo dall'URL
 - Lo aggiunge dinamicamente alla pagina
 - Il browser dell'utente interpreta lo script malevolo